DOCUMENTATION PROJET · VERSION 1.0

GUDLFT

Documentation fonctionnelle et technique

Utilité des fonctionnalités ajoutées et résolution des bugs

DATE
25 octobre 2026

STATUT
Version livrée — prête pour revue

PÉRIMÈTRE
Phase 1, Phase 2, administration

TECHNOLOGIE
Flask · Jinja2 · JSON

Projet GUDLFT
Application de réservation de places de compétitions pour clubs de powerlifting.

Tests : 94 · Couverture : 100 %
Flake8 : 0 erreur · Locust : 0 échec

FICHE DOCUMENTAIRE

Résumé exécutif

GUDLFT est une application Flask de démonstration permettant aux secrétaires de clubs de réserver des places de compétitions avec les points accumulés par leur club. Le projet livré ne se contente pas de corriger les bugs de l'application initiale : il ajoute un classement public, une interface d'administration complète, la gestion des points, le pilotage des inscriptions et un classement par compétition.

Ce document explique, fonctionnalité par fonctionnalité, **pourquoi** chaque ajout est utile, **comment** il fonctionne et **par quels tests** il est vérifié. Il détaille ensuite la résolution des bugs, avec pour chaque cas le symptôme observé, la cause racine, la correction apportée et la preuve de non-régression.

Le projet est livré avec **94 tests automatisés**, une couverture de **100 %** de `server.py`, **0 erreur Flake8** et un test de performance Locust sans échec. Toutes les branches de travail sont mergées dans `main` avec des commits de merge explicites, et la branche `QA` est alignée sur `main` pour la revue finale.

94 tests automatisés, tous verts	**100 %** couverture de server.py	**0** erreur Flake8 / PEP8
0 échec sur le test Locust	**13** corrections de bugs documentées	**8** groupes de fonctionnalités ajoutées

> **Lecture rapide.** Pour comprendre la valeur du projet, lire la section 3. Pour vérifier la résolution des bugs, lire la section 5. Pour la conformité au cahier des charges et les preuves de qualité, lire les sections 6 et 7.

Ce que ce document couvre

- Le besoin métier, le périmètre livré et les contraintes du projet.
- L'architecture Flask, les fichiers JSON et les règles métier.
- Les fonctionnalités : classement public, classement par compétition, administration, points, inscriptions et persistance.
- Les scénarios de bout en bout et les codes HTTP utilisés.
- La résolution détaillée des bugs, avec cause, correction et tests.
- La traçabilité des exigences, les résultats de tests, la performance et les limites connues.
- Les annexes : routes, données, inventaire des tests, commandes et glossaire.

> **Périmètre et avertissement.** Ce document décrit une application de démonstration / formation. Les données JSON, le mot de passe administrateur par défaut et la clé de session ne doivent pas être utilisés tels quels en production. Les recommandations de durcissement sont détaillées dans la section 8.

NAVIGATION

Sommaire

1 Contexte, besoin et périmètre

2 Architecture, données et règles métier

3 Fonctionnalités ajoutées

- 3.1 Tableau public des points
- 3.2 Classement par compétition
- 3.3 Interface d’administration
- 3.4 Gestion des points
- 3.5 Ouverture et fermeture des inscriptions
- 3.6 Persistance des réservations
- 3.7 Expérience club et messages
- 3.8 Qualité, performance et documentation

4 Scénarios de bout en bout

5 Résolution des bugs

6 Traçabilité des exigences

7 Qualité, performance et sécurité

8 Limites et pistes d’amélioration

9 Conclusion

A Annexe A — Routes de l’application

B Annexe B — Modèle de données

C Annexe C — Inventaire des tests

D Annexe D — Commandes utiles

E Annexe E — Glossaire

F Annexe F — Comptes de démonstration

Convention de lecture. Les blocs `comme ceci` désignent des routes, fichiers, fonctions ou commandes. Les encadrés bleus expliquent l’utilité métier ; les encadrés verts signalent une preuve ou un résultat ; les encadrés orange signalent une limite ou une vigilance.

SECTION 1

1. Contexte, besoin et périmètre

1.1 Le besoin métier

Les compétitions de powerlifting sont organisées localement et régionalement. Les athlètes ne réservent pas directement auprès de l'organisateur : ce sont les secrétaires de clubs qui inscrivent les athlètes et réservent les places. Chaque club accumule des points en organisant et en faisant vivre des compétitions. Ces points servent de monnaie d'échange : **un point correspond à une place** dans une compétition future.

Le système doit garantir trois choses : que les clubs disposent d'un solde fiable, qu'ils ne puissent pas réserver plus de places que disponible, et qu'aucun club ne puisse accaparer une compétition. La limite de **12 places par club et par compétition** assure l'équité entre les clubs. Enfin, les points de chaque club doivent être consultables publiquement, sans connexion, pour éviter les soupçons de favoritisme.

> **Règle fondatrice.** Un club ne peut réserver une place que s'il possède le point correspondant. Les points sont déduits au moment de la réservation, et les places restantes de la compétition sont diminuées dans le même mouvement.

1.2 Le problème initial

L'application de démonstration fournie au départ permettait de visualiser des clubs et des compétitions, mais elle comportait des bugs critiques : certains e-mails inconnus provoquaient une erreur serveur, les points n'étaient pas toujours déduits, les réservations négatives ou supérieures à 12 places pouvaient passer, les compétitions passées restaient réservables et un club inconnu pouvait faire planter la page. Le classement public et l'interface d'administration n'existaient pas.

Le projet consistait donc à la fois à **fiabiliser l'existant** et à **ajouter les fonctionnalités attendues** par les organisateurs, sans casser le parcours simple du secrétaire de club.

1.3 Objectifs du projet

- **Fiabiliser** les réservations et empêcher toute incohérence de points ou de places.
- **Sécuriser** les accès administrateur et les opérations sensibles.
- **Rendre autonomes** les organisateurs grâce à une interface d'administration.
- **Apporter de la transparence** avec un classement public accessible sans connexion.
- **Donner du contexte** au classement avec une vue par compétition.
- **Prouver la qualité** avec des tests, une couverture élevée, Flake8 et Locust.
- **Documenter et tracer** les décisions, les branches et les corrections.

1.4 Périmètre livré

Bloc	Contenu livré	Statut
Phase 1	Correctifs de connexion, réservation, validation, points, dates, HTML, session et PEP8.	Livré
Phase 2	Classement public des points, accessible sans connexion, en lecture seule.	Livré
Classement par compétition	Sélecteur de compétition, classement par places réservées, date, places restantes et état des inscriptions.	Livré

Bloc	Contenu livré	Statut
Administration	Connexion admin, tableau de bord, CRUD clubs et compétitions.	Livré
Gestion des points	Crédit, débit, motif, contrôle du solde et historique horodaté.	Livré
Inscriptions	Ouverture et fermeture par compétition, blocage des réservations fermées.	Livré
Qualité	94 tests, couverture 100 %, Flake8 0, Locust sans échec, README et rapports HTML.	Livré
Git	Branches de travail, commits de merge explicites, branche QA alignée sur main.	Livré

1.5 Contraintes et critères d'acceptation

Contrainte	Critère	Résultat
Framework	Application Flask avec templates Jinja2.	Respecté
Stockage	Fichiers JSON légers, lisibles et modifiables.	Respecté
Performance	Liste des compétitions en moins de 5 s ; mise à jour des points en moins de 2 s.	Respecté
Charge	Test Locust avec 6 utilisateurs.	Respecté
Tests	Au moins 60 % de couverture.	100 %
Documentation	README avec installation, lancement, comptes, tests et conventions.	Respecté
Git	Une branche par correctif ou fonctionnalité, puis merge dans main.	Respecté
Revue	Branche QA disponible pour la revue finale.	Respecté

1.6 Niveau de complétion

Toutes les exigences fonctionnelles identifiées dans le cahier des charges sont couvertes, ainsi que des fonctionnalités d'administration demandées en complément. Le projet est donc **fonctionnellement complet**. La recette finale, les derniers essais manuels et les éventuels retours de l'organisme de formation restent à réaliser.

Points vérifiés avant livraison

✓ Tous les tests passent. ✓ La couverture de `server.py` est de 100 %. ✓ Flake8 ne signale aucune erreur. ✓ Le test Locust ne présente aucun échec. ✓ Les branches sont mergées. ✓ Le README est à jour. ✓ L'application répond en HTTP 200 sur les pages principales.

SECTION 2

2. Architecture, données et règles métier

2.1 Vue d’ensemble

L’application suit une architecture volontairement simple : un serveur Flask unique, des templates Jinja2 et trois fichiers JSON. Cette simplicité est adaptée à un projet de démonstration : elle rend le code lisible, les données inspectables et les tests faciles à exécuter.



2.2 Responsabilités principales

Composant	Responsabilité
`server.py`	Routes Flask, validation des entrées, règles métier, chargement et sauvegarde des données, sessions admin.
`templates/`	Pages club, page publique, formulaires et interface d’administration.
`clubs.json`	Nom, e-mail, points, historique des ajustements et places réservées par compétition.
`competitions.json`	Nom, date, nombre de places restantes et état des inscriptions.
`admin.json`	Comptes administrateur de démonstration.
`tests/`	Tests automatisés avec le client Flask et pytest.
`rapport_flake8.html`	Rapport HTML de conformité PEP8.
`rapport_locust.html`	Rapport HTML du test de charge.

2.3 Modèle de données

Les données sont volontairement plates et lisibles. Le champ `points_history` conserve la trace des ajustements de points. Le champ `booked` conserve, pour chaque club, le nombre de places déjà réservées par compétition. Le champ `registration_open` pilote l’ouverture ou la fermeture des inscriptions.

Exemple de club :

```
{
  "name": "Simply Lift",
  "email": "john@simplylift.co",
  "points": 13,
  "points_history": [
    {
      "date": "2026-10-25 10:15:00",
      "admin": "Super Admin",
      "amount": 5,
      "reason": "Bonus partenaire",
      "balance": 18
    }
  ],
  "booked": {
    "Future Open": 2
  }
}
```

Exemple de compétition :

```
{
  "name": "Future Open",
  "date": "2099-06-15 10:00:00",
  "numberOfPlaces": 25,
  "registration_open": true
}
```

Exemple de compte administrateur :

```
[
  {
    "email": "admin@gudlft.com",
    "password": "<mot de passe>",
    "name": "Super Admin"
  }
]
```

2.4 Règles métier

Règle	Application dans le code	Conséquence si non respectée
1 point = 1 place	Déduction de `placesRequired` sur `club['points']`.	Solde incohérent, réservation gratuite.
Maximum 12 places par club et par compétition	Cumul dans `club['booked'][competition]` et blocage si le total dépasse 12.	Un club peut accaparer la compétition.
Pas de surréservation	Comparaison avec `competition['numberOfPlaces']`.	Plus de places vendues que disponibles.
Pas de réservation dans le passé	Comparaison de la date de compétition avec `datetime.now()`.	Inscriptions sur des événements terminés.
Pas de réservation si inscriptions fermées	Contrôle de `registration_open` dans `/book` et `/purchasePlaces`.	Contournement de la date limite.
Pas de solde négatif	Contrôle des points avant réservation et contrôle du débit admin.	Dette de points impossible à régulariser.

Règle	Application dans le code	Conséquence si non respectée
Montant d'ajustement positif	Crédit ou débit d'un entier strictement supérieur à 0.	Opérations nulles ou ambiguës.
Historique borné	Maximum 50 entrées conservées par club.	Fichier JSON qui grossit indéfiniment.

2.5 Cycle de chargement et de sauvegarde

Au démarrage, `loadClubs()` et `loadCompetitions()` lisent les fichiers JSON avec l'encodage `utf-8-sig`, puis normalisent les types : les points et les places sont convertis en entiers, et `registration_open` est converti en booléen. Les routes métier modifient les objets en mémoire, puis appellent `saveClubs()` et `saveCompetitions()` lorsque la modification doit être persistée.

Les sauvegardes utilisent `ensure_ascii=False` pour conserver les accents lisibles dans les fichiers. Les tests utilisent une fixture `autouse` qui remplace les fonctions de sauvegarde par des fonctions vides : les tests ne modifient donc jamais les fichiers JSON réels du projet.

2.6 Pourquoi JSON, et quelles limites ?

JSON est adapté à un prototype : aucun serveur de base de données à installer, données faciles à inspecter et à versionner, tests simples. En revanche, JSON n'offre ni transaction, ni verrouillage concurrent, ni requêtes, ni contrôle d'intégrité. Pour une mise en production réelle, il faudrait migrer vers une base de données et ajouter des transactions.

2.7 Codes HTTP utilisés

Code	Signification dans GUDLFT	Exemple
200	Page ou opération réussie.	Connexion valide, réservation acceptée, classement affiché.
302	Redirection après une action.	Déconnexion, ajout ou suppression admin.
400	Requête invalide ou règle métier non respectée.	Places négatives, compétition passée, points insuffisants, plus de 12 places.
401	Identification club impossible.	E-mail inconnu ou vide.
403	Action interdite.	Réservation sur une compétition dont les inscriptions sont fermées.
404	Ressource introuvable.	Club ou compétition inexistant.

SECTION 3

3. Fonctionnalités ajoutées

Les fonctionnalités ajoutées répondent à trois promesses : rendre le système plus transparent, donner aux organisateurs les moyens d'administrer la plateforme sans toucher aux fichiers JSON, et garantir que les réservations restent fiables dans le temps.

Fonctionnalité	Utilité principale	Public	Statut
3.1 Tableau public des points	Transparence et comparaison des clubs.	Tous	Livré
3.2 Classement par compétition	Donner du contexte et mesurer la participation.	Tous	Livré
3.3 Interface d'administration	Gérer clubs et compétitions sans JSON.	Admin	Livré
3.4 Gestion des points	Créditer, débiter et tracer les ajustements.	Admin	Livré
3.5 Inscriptions	Ouvrir ou fermer les réservations à volonté.	Admin	Livré
3.6 Persistance	Conserver les réservations après redémarrage.	Système	Livré
3.7 Expérience club	Messages clairs, erreurs propres, parcours accessible.	Clubs	Livré
3.8 Qualité et documentation	Prouver la fiabilité et faciliter la reprise.	Équipe / correcteur	Livré

3.1 · TRANSPARENCE

Public | Lecture seule | Sans connexion

Tableau public des points

Utilité. Donner à tous les clubs une vision commune et vérifiable des points disponibles, sans connexion ni intervention de l'administrateur. Ce tableau répond directement au besoin de transparence exprimé dans le cahier des charges, après des soupçons de favoritisme.

FONCTIONNEMENT

- Page exposée sur `/displayBoard` et sur l'alias `/points`.
- Tous les clubs et leurs points sont affichés, triés par points décroissants puis par nom.
- Les trois premiers clubs sont mis en avant avec une médaille visuelle.
- La page est strictement en lecture seule : aucun formulaire de réservation, aucune action admin.

BÉNÉFICES

- Confiance accrue entre clubs et organisateurs.
- Comparaison immédiate des soldes, sans demander un export.
- Réduction du risque de contestation ou de favoritisme.
- Accessibilité totale : aucun compte requis.

PREUVES ET TESTS

- Accès public : `test_display_board_public_access`, `test_points_alias_access`.
- Tri par points : `test_display_board_clubs_sorted_by_points`.
- Présence de tous les clubs : `test_display_board_renders_all_clubs_and_points`.
- Liens depuis l'accueil et le tableau de bord club : `test_index_contains_display_board_link`, `test_welcome_contains_display_board_link`.

Fichiers : `server.py` (route `displayBoard`), `templates/display_board.html`, `static/css/style.css`.

3.2 · CONTEXTE SPORTIF

Sélecteur | Places réservées | Temps réel

Classement par compétition

Utilité. Le classement général ne dit pas qui participe réellement à une compétition. Le classement par compétition apporte ce contexte : il montre quels clubs ont réservé des places et dans quel ordre, ce qui aide les clubs à se situer et les organisateurs à suivre la participation.

FONCTIONNEMENT

- Un sélecteur propose le classement général ou une compétition précise.
- La sélection utilise le paramètre `?competition=Future%20Open`.
- Le classement s'appuie sur `club['booked'][competition]`.
- Seuls les clubs ayant réservé au moins une place apparaissent.
- Tri : places réservées décroissantes, puis points du club, puis nom.
- L'en-tête affiche la date, les places restantes et l'état des inscriptions.
- Un état vide explique qu'aucune place n'a encore été réservée.

BÉNÉFICES

- Lecture plus juste de la participation réelle.
- Suivi de la demande par compétition.
- Meilleure préparation logistique pour l'organisateur.
- Cohérence avec le classement général, sans mélanger les deux notions.

PREUVES ET TESTS

- Présence du sélecteur : `test_display_board_has_competition_selector`.
- Classement par places réservées : `test_display_board_competition_ranking`.
- État vide : `test_display_board_competition_without_booking`.
- Compétition inconnue : retour au classement général avec `test_display_board_unknown_competition_shows_general`.

Fichiers : `server.py`, `templates/display_board.html`, `clubs.json` (champ `booked`), `competitions.json`.

3.3 · AUTONOMIE

Session protégée | CRUD | Tableau de bord

Interface d’administration

Utilité. Permettre à l’organisateur de gérer les clubs et les compétitions depuis une interface web, sans éditer les fichiers JSON à la main. Cela réduit les erreurs de saisie, centralise les actions et rend l’application utilisable par une personne non technique.

FONCTIONNEMENT

- Connexion sur `/admin/login`, session Flask et décorateur `admin_required`.
- Tableau de bord avec nombre de clubs, nombre de compétitions et total des points.
- Liste, ajout, modification et suppression des clubs.
- Liste, ajout, modification et suppression des compétitions.
- Validation des champs : nom et e-mail requis, points et places entiers.
- Confirmation JavaScript avant suppression d’un club ou d’une compétition.
- Déconnexion admin avec purge de la session.

BÉNÉFICES

- Autonomie complète de l’organisateur.
- Moins de risques de casser le JSON à la main.
- Actions regroupées et cohérentes dans une seule interface.
- Base saine pour de futures fonctions d’administration.

PREUVES ET TESTS

- Protection des routes : `test_admin_routes_require_login` (11 routes testées).
- Connexion et déconnexion : `test_admin_login_success`, `test_admin_logout`.
- CRUD clubs et compétitions : tests `test_admin_add_club`, `test_admin_edit_club`, `test_admin_delete_club`, `test_admin_add_competition`, etc.
- Validation et erreurs : `test_admin_add_club_invalid_points`, `test_admin_edit_competition_invalid_places`.

Fichiers : `server.py`, `templates/admin/`, `static/css/style.css`, `admin.json`.

3.4 · TRAÇABILITÉ

Crédit / débit | Historique | Solde protégé

Gestion des points des clubs

Utilité. Les points sont la monnaie du système : ils doivent pouvoir être corrigés après un bonus, une erreur de saisie ou une régularisation. L'administration des points évite de modifier le JSON à la main et conserve une trace horodatée de chaque opération.

FONCTIONNEMENT

- Page dédiée par club : `/admin/clubs/<club>/points`.
- Crédit ou débit d'un montant entier strictement supérieur à 0.
- Motif facultatif ; libellé par défaut « Ajustement manuel ».
- Contrôle du solde : un débit ne peut pas rendre le solde négatif.
- Historique : date, administrateur, montant, motif et solde après opération.
- Maximum de 50 entrées conservées par club ; 20 affichées dans la fiche.

BÉNÉFICES

- Auditabilité des ajustements de points.
- Confiance des clubs dans la gestion du solde.
- Correction rapide sans risque de casser le fichier JSON.
- Respect de la règle « pas de solde négatif ».

PREUVES ET TESTS

- Page et historique : `test_admin_points_page_displays_club_and_history`.
- Crédit : `test_admin_credit_points` ; débit : `test_admin_debit_points`.
- Solde négatif refusé : `test_admin_debit_cannot_make_balance_negative`.
- Montant invalide ou nul : `test_admin_points_invalid_amount`, `test_admin_points_amount_must_be_positive`.
- Motif par défaut et opération inconnue : `test_admin_points_default_reason`, `test_admin_points_invalid_operation`.

Fichiers : `server.py` (`recordPointsChange`, `adminClubPoints`), `templates/admin/club_points.html`, `clubs.json`.

SECTION 3 · SUITE

3. Fonctionnalités ajoutées (suite)

3.5 · PILOTAGE

Par compétition | Blocage 403 | Sans JSON

Ouverture et fermeture des inscriptions

Utilité. Une compétition peut être complète, reportée ou simplement clôturée à une date limite. L'organisateur doit pouvoir fermer les inscriptions sans supprimer la compétition ni modifier le code. Les clubs voient immédiatement que la réservation n'est plus possible.

FONCTIONNEMENT

- Chaque compétition possède un booléen `registration_open`, vrai par défaut.
- Bouton **Fermer** ou **Ouvrir** dans la liste admin.
- Case à cocher dans les formulaires d'ajout et de modification.
- La route `/book` et la route `/purchasePlaces` renvoient HTTP 403 si les inscriptions sont fermées.
- Le tableau de bord club affiche « Inscriptions fermées » et masque le lien de réservation.
- Toute modification est sauvegardée dans `competitions.json`.

BÉNÉFICES

- Contrôle opérationnel immédiat, sans intervention technique.
- Respect de la date limite et de la capacité réelle.
- Messages cohérents entre l'admin et les clubs.
- Aucune suppression de données nécessaire.

PREUVES ET TESTS

- Bascule fermeture / réouverture : `test_admin_toggle_registration_closes_then_reopens`.
- Compétition inconnue : `test_admin_toggle_unknown_competition`.
- Blocage de la page et de l'achat : `test_closed_competition_blocks_booking_page`, `test_closed_competition_blocks_purchase`.
- Affichage côté club : `test_welcome_shows_closed_registration`.
- Ajout par défaut ouvert, ajout fermé et modification : trois tests dédiés.

Fichiers : `server.py`, `templates/admin/competitions.html`, `templates/admin/add_competition.html`, `templates/admin/edit_competition.html`, `templates/welcome.html`, `competitions.json`.

3.6 · FIABILITÉ

Sauvegarde | Après redémarrage | Champ booked

Persistance des réservations et du classement

Utilité. Une réservation doit survivre au redémarrage du serveur. Sans persistance, les points et les places étaient bien modifiés en mémoire, mais les fichiers JSON restaient inchangés : le classement par compétition et l'historique de participation devenaient faux après un redémarrage.

FONCTIONNEMENT

- Chaque club possède un dictionnaire `booked` : compétition → places réservées.
- Après une réservation validée, `saveClubs()` et `saveCompetitions()` sont appelés.
- Les points, les places restantes et le compteur `booked` sont ainsi persistés ensemble.
- La fixture `prevent_disk_writes` protège les tests des écritures réelles.

BÉNÉFICES

- Aucune perte de réservation après redémarrage.
- Classement par compétition fiable dans le temps.
- Base nécessaire au respect de la limite de 12 places.
- Comportement cohérent entre mémoire et fichiers.

PREUVES ET TESTS

- Mise à jour de `booked` et appels de sauvegarde : `test_reservation_enregistre_places_et_donnees`.
- Sauvegarde JSON réelle : `test_save_functions_write_json`.
- Maintien des types entiers après réservation : `test_reservation_maintient_types_entiers`.

Fichiers : `server.py` (`saveClubs`, `saveCompetitions`, `purchasePlaces`), `clubs.json`, `competitions.json`, `tests/conftest.py`.

3.7 · EXPÉRIENCE UTILISATEUR

Messages flash | Accessibilité | Erreurs propres

Expérience club et messages

Utilité. Un utilisateur doit comprendre ce qui se passe : pourquoi une réservation échoue, pourquoi un e-mail est inconnu, si les inscriptions sont ouvertes ou si une opération a réussi. Les messages clairs et les codes HTTP adaptés évitent les pages blanches et les erreurs serveur.

FONCTIONNEMENT

- Messages flash pour les succès, erreurs et confirmations.
- Codes HTTP explicites : 400, 401, 403 et 404 selon le cas.
- Formulaire de réservation avec `min="1"`, `max="12"` et `required`.
- HTML nettoyé : plus de `</br>`, plus d'identifiants vides.
- Tableau de bord club avec état des inscriptions et liens utiles.
- Confirmation avant suppression côté admin.

BÉNÉFICES

- Moins de saisies invalides grâce à la validation navigateur.
- Meilleure accessibilité des formulaires.
- Diagnostic plus rapide en cas d'erreur.
- Parcours plus rassurant pour les secrétaires de clubs.

PREUVES ET TESTS

- Formulaires et HTML : `test_index_template_attributes`, `test_booking_template_attributes`, `test_welcome_template_clean_html`.
- Déconnexion et message : `test_logout_message_flash`, `test_logout_purge_session`.
- Erreurs de réservation et codes HTTP : tests `test_saisie_places.py`, `test_limite_places.py` et `test_club_competition_inconnus.py`.

Fichiers : `templates/index.html`, `templates/welcome.html`, `templates/booking.html`, `templates/display_board.html`, `server.py`.

3.8 · PREUVES

94 tests | 100 % | Locust | Git

Qualité, performance et documentation

Utilité. Une fonctionnalité n'est livrable que si elle est testée, mesurée et documentée. Cette couche de qualité permet au correcteur de vérifier les résultats, à l'équipe de reprendre le projet et au commanditaire d'avoir confiance dans la livraison.

FONCTIONNEMENT

- 94 tests automatisés avec pytest et client Flask.
- Couverture de `server.py` à 100 %.
- Flake8 à 0 erreur sur le code, les tests et les scripts.
- Locust : 6 utilisateurs, 30 secondes, 97 requêtes, 0 échec.
- README avec installation, lancement, comptes, tests, performance et workflow Git.
- Rapports HTML : `rapport_flake8.html`, `rapport_locust.html`, `htmlcov/index.html`.
- Branches de travail, commits de merge explicites et branche QA alignée sur main.

BÉNÉFICES

- Preuve objective de non-régression.
- Confiance dans les performances et la stabilité.
- Reprise et maintenance facilitées.
- Historique Git lisible et traçable.

PREUVES ET RÉSULTATS

- Tests : 94/94, couverture 100 %, Flake8 0.
- Locust : moyenne 2 ms, p95 4 ms, maximum 7 ms.
- Seuils du cahier des charges : liste < 5 s, mise à jour des points < 2 s.
- Branches : toutes mergées ; `QA` = `main`.

Fichiers : `tests/`, `README.md`, `rapport_flake8.html`, `rapport_locust.html`, `htmlcov/index.html`.

SECTION 4

4. Scénarios de bout en bout

Les scénarios ci-dessous montrent comment les fonctionnalités se combinent dans un parcours réel. Ils peuvent servir de support de démonstration ou de recette manuelle.

4.1 Scénario A — Un club réserve trois places

1. Le secrétaire ouvre `/` et saisit l'e-mail de son club, par exemple `john@simplylift.co`.
2. Le tableau de bord affiche le solde de points, les compétitions et l'état des inscriptions.
3. Le secrétaire clique sur **Future Open**. La page de réservation affiche les places restantes et un champ limité à 12.
4. Il saisit 3 places puis valide. Le serveur vérifie le club, la compétition, la date, l'état des inscriptions, le nombre positif, le cumul de 12, les places restantes et les points.
5. Les points sont diminués de 3, les places restantes de 3, et `booked["Future Open"]` passe à 3. Les fichiers JSON sont sauvegardés.
6. Le message « Great-booking complete! » confirme l'opération. Le classement par compétition affiche désormais Simply Lift avec 3 places réservées.

4.2 Scénario B — Un admin crédite des points

1. L'administrateur se connecte sur `/admin/login`.
2. Il ouvre **Admin > Clubs** puis clique sur **Points** pour Simply Lift.
3. Il choisit **Crédit**, saisit 5 et indique le motif « Bonus partenaire ».
4. Le solde passe de 13 à 18. Une entrée d'historique est créée avec la date, l'admin, le montant, le motif et le nouveau solde.
5. Le fichier `clubs.json` est sauvegardé et le classement public reflète immédiatement le nouveau solde.

4.3 Scénario C — L'admin ferme les inscriptions

1. L'admin ouvre **Admin > Compétitions**.
2. Il clique sur **Fermer** pour Future Open et confirme l'action.
3. Le badge passe à « Fermées » et `registration_open` devient `false` dans `competitions.json`.
4. Le tableau de bord club affiche « Inscriptions fermées » et masque le lien de réservation.
5. Une tentative directe sur `/book/...` ou `/purchasePlaces` renvoie HTTP 403 sans modifier les points ni les places.

4.4 Scénario D — Un visiteur consulte le classement par compétition

1. Le visiteur ouvre `/displayBoard` sans se connecter.
2. Il sélectionne **Future Open** dans le sélecteur de compétition.
3. L'en-tête affiche la date, les places restantes et l'état des inscriptions.
4. Le tableau classe uniquement les clubs ayant réservé des places, par nombre de places décroissant.
5. Si aucune réservation n'existe, un état vide explique la situation au lieu d'afficher un tableau trompeur.

4.5 Scénario E — Les erreurs sont propres

Action	Résultat attendu	Code HTTP
E-mail inconnu	Message « adresse e-mail non trouvée », pas de crash.	401
Club ou compétition inexistant	Message explicite, retour à une page valide.	404
Compétition passée	Réservation refusée, aucune modification.	400
Places négatives, nulles ou non numériques	Message de validation, aucune modification.	400
Plus de 12 places ou surréservation	Message de limite, aucune modification.	400
Points insuffisants	Message de solde insuffisant, aucune modification.	400
Inscriptions fermées	Message de fermeture, aucun accès au formulaire.	403

SECTION 5

5. Résolution des bugs

Chaque bug a été traité avec la même méthode : reproduire le problème, isoler la cause racine, corriger au plus près, ajouter un test de non-régression, puis vérifier l'ensemble de la suite et Flake8. Cette section documente les 13 corrections de bugs et les 2 améliorations de robustesse qui ont un impact direct sur la fiabilité du logiciel.

5.1 Méthode de correction

1. **Reproduire.** Le bug est reproduit avec le client Flask ou une requête HTTP réelle, pour observer le comportement exact.
2. **Isoler.** La cause est recherchée dans la route, le template ou le chargement des données, pas seulement dans le symptôme visible.
3. **Corriger.** La correction reste locale et respecte les règles métier existantes.
4. **Tester.** Un test de non-régression vérifie le comportement attendu et les effets de bord : points, places, fichiers et codes HTTP.
5. **Vérifier.** La suite complète, la couverture et Flake8 sont relancés avant le commit.
6. **Documenter.** Le correctif est nommé, commité sur une branche dédiée, puis mergé avec un commit de merge explicite.

5.2 Synthèse des corrections

ID	Bug ou amélioration	Gravité	Correction principale	Tests
B1	Connexion avec un e-mail inconnu	Critique	HTTP 401 et message propre	`test_connexion.py`
B2	Club ou compétition inconnus	Critique	HTTP 404 et message propre	`test_club_competition_inconnus.py`
B3	Points non déduits après réservation	Critique	Déduction effective du solde	`test_points.py`
B4	Réservation avec points insuffisants	Majeur	Blocage HTTP 400	`test_points.py`
B5	Plus de 12 places par compétition	Majeur	Cumul dans `booked` et blocage	`test_limite_places.py`
B6	Surréservation d'une compétition	Majeur	Contrôle des places restantes	`test_limite_places.py`
B7	Places négatives, nulles ou non numériques	Majeur	Validation HTTP 400	`test_saisie_places.py`
B8	Réservation d'une compétition passée	Majeur	Contrôle de la date	`test_competitions_passees.py`
B9	HTML invalide et formulaires incomplets	Moyen	Templates nettoyés et accessibles	`test_templates_html.py`
B10	Types JSON incohérents	Majeur	Normalisation en entiers et booléens	`test_types_donnees.py`

ID	Bug ou amélioration	Gravité	Correction principale	Tests
B11	Déconnexion sans purge de session	Majeur	`session.clear()` et message flash	`test_deconnexion.py`
B12	Non-conformités PEP8 / Flake8	Mineur	Refactorisation et style	`rapport_flake8.html`
B13	BOM UTF-8 dans `admin.json`	Moyen	Lecture avec `utf-8-sig`	`test_admin.py`
B14	Réservations non persistées	Amélioration	Sauvegarde et champ `booked`	`test_points.py`
B15	Classement général non trié	Amélioration	Tri par points puis nom	`test_points_display.py`

5.3 Avant / après

Situation	Avant	Après
E-mail inconnu	Erreur serveur / traceback	HTTP 401 et message lisible
Club ou compétition inconnu	Erreur serveur	HTTP 404 et message lisible
Places négatives ou nulles	Saisie acceptée ou comportement incohérent	HTTP 400, aucune modification
Plus de 12 places	Réservation acceptée	HTTP 400, limite expliquée
Compétition passée	Réservation possible	HTTP 400, réservation refusée
Inscriptions fermées	Aucun état, réservation possible	HTTP 403 et lien masqué
Points du club	Solde inchangé après réservation	Points déduits et sauvegardés
Surréservation	Plus de places vendues que disponibles	HTTP 400, capacité protégée
Déconnexion	Session potentiellement conservée	Session vidée et message de confirmation
Classement général	Ordre du fichier JSON	Tri par points décroissants puis nom

B1 Connexion avec un e-mail inconnu

CRITIQUE

SYMPTÔME

La saisie d'une adresse e-mail absente de `clubs.json` provoquait une erreur serveur et affichait un traceback. L'utilisateur ne comprenait pas si l'adresse était mauvaise ou si l'application était en panne.

CAUSE RACINE

La route `/showSummary` construisait une liste de clubs correspondants, puis prenait directement le premier élément avec `[0]` sans vérifier que la liste était non vide.

CORRECTION

Un contrôle explicite renvoie désormais HTTP 401 avec le message « Désolé, cette adresse e-mail n'a pas été trouvée. » et réaffiche la page de connexion. Le cas de l'e-mail vide est traité de la même manière.

IMPACT

Plus de crash, message compréhensible, comportement professionnel et code HTTP conforme à un échec d'identification.

Tests : `test_connexion_email_inconnu`, `test_connexion_email_vide`, `test_connexion_reussie` — **Fichiers** : `server.py`, `templates/index.html`.

B2 Club ou compétition inconnus

CRITIQUE

SYMPTÔME

Une URL de réservation contenant un club ou une compétition inexistante provoquait une erreur serveur. Le formulaire `/purchasePlaces` pouvait également planter avec des noms invalides.

CAUSE RACINE

Les routes `/book` et `/purchasePlaces` prenaient le premier résultat d'une recherche sans vérifier qu'un résultat existait.

CORRECTION

Des contrôles renvoient HTTP 404 avec « Club introuvable. » ou « Compétition introuvable. ». La page d'accueil est réaffichée proprement et aucun point ni aucune place n'est modifié.

IMPACT

Robustesse des routes, meilleure sécurité et diagnostic immédiat pour l'utilisateur.

Tests : `test_book_club_inconnu`, `test_book_competition_inconnue`, `test_purchase_club_inconnu`, `test_purchase_competition_inconnue` et test de page valide — **Fichiers :** `server.py`.

B3 Points non déduits après réservation

CRITIQUE

SYMPTÔME

La réservation affichait un succès et diminuait les places de la compétition, mais le solde de points du club restait inchangé. Un club pouvait donc réserver gratuitement.

CAUSE RACINE

La route `purchasePlaces` mettait à jour les places restantes, mais n'affectait jamais la nouvelle valeur au champ `points` du club.

CORRECTION

Le solde est maintenant calculé puis réaffecté explicitement : `club['points'] = int(club['points']) - placesRequired`. La valeur est convertie en entier pour éviter les concaténations de chaînes.

IMPACT

Intégrité de la monnaie du système, équité entre clubs et cohérence entre les points affichés et les places réservées.

Tests : `test_deduction_points_apres_reservation`, `test_reservation_maintient_types_entiers` — **Fichiers :** `server.py`, `clubs.json`.

B4 Réservation avec points insuffisants

MAJEUR

SYMPTÔME

Un club pouvait réserver plus de places que son solde de points, ce qui produisait un solde négatif ou une incohérence difficile à corriger.

CAUSE RACINE

Aucun contrôle ne comparait le nombre de places demandées au solde de points avant la déduction.

CORRECTION

La route vérifie `placesRequired > int(club['points'])` et renvoie HTTP 400 avec un message expliquant le solde insuffisant. Aucune donnée n'est modifiée.

IMPACT

Impossible de créer une dette de points, règle métier « un point = une place » réellement appliquée.

Tests : `test_reservation_points_insuffisants` — **Fichiers :** `server.py`, `templates/booking.html`.

B5 Plus de 12 places par compétition

MAJEUR

SYMPTÔME

Un club pouvait réserver 13 places ou plus, en une seule fois ou en plusieurs réservations successives, ce qui permettait d'accaparer une compétition.

CAUSE RACINE

La limite n'était pas stockée par club et par compétition. Le serveur ne connaissait pas le cumul déjà réservé.

CORRECTION

Le champ `booked` conserve le nombre de places déjà réservées. Le contrôle `already_booked + placesRequired > 12` bloque la réservation avec HTTP 400 et un message indiquant la limite de 12 places.

IMPACT

Équité entre clubs, meilleure répartition des places et règle du cahier des charges appliquée sur toute la durée de la compétition.

Tests : `test_reservation_superieure_a_12_places_refusee`, `test_reservation_cumulee_superieure_a_12_places_refusee` — **Fichiers :** `server.py`, `clubs.json`.

SECTION 5 · SUITE

5. Résolution des bugs (suite)

B6 Surréservation d'une compétition

MAJEUR

SYMPTÔME

Le nombre de places demandées pouvait dépasser le nombre de places encore disponibles dans la compétition. La compétition affichait alors un stock négatif ou incohérent.

CAUSE RACINE

La route `purchasePlaces` ne comparait pas la demande au champ `numberOfPlaces` avant de le décrémenter.

CORRECTION

Un contrôle compare désormais la demande au stock restant. Si la demande est supérieure, la route renvoie HTTP 400 avec un message sur les « places disponibles ». Les points et les places ne sont pas modifiés.

IMPACT

La capacité de la compétition est protégée, les organisateurs ne peuvent pas survendre et les clubs ne reçoivent pas de confirmation trompeuse.

Tests : `test_surreservation_places_competition_refusee` — **Fichiers :** `server.py`, `competitions.json`.

B7 Places négatives, nulles ou non numériques

MAJEUR

SYMPTÔME

Une saisie de `-5` pouvait être acceptée et produire un comportement absurde, une saisie de `0` créait une réservation vide, et une saisie texte pouvait provoquer une erreur serveur.

CAUSE RACINE

La valeur du champ `places` était convertie en entier sans gestion d'erreur, et aucune borne minimale n'était vérifiée côté serveur.

CORRECTION

La conversion est protégée par `try/except`, puis la valeur doit être strictement positive. Les cas négatif, nul et non numérique renvoient HTTP 400 avec un message de validation, sans aucune modification des données.

IMPACT

Intégrité des réservations, disparition des erreurs 500 et validation serveur qui complète la validation HTML5.

Tests : `test_reservation_places_negatives_refusee`, `test_reservation_zero_place_refusee`, `test_reservation_places_non_numeriques_refusee` — **Fichiers :** `server.py`, `templates/booking.html`.

B8 Réservation d’une compétition passée

MAJEUR

SYMPTÔME

Un club pouvait ouvrir et valider une réservation pour une compétition dont la date était déjà passée, par exemple Spring Festival ou Fall Classic.

CAUSE RACINE

La date de la compétition était affichée mais jamais comparée à la date courante dans les routes de réservation.

CORRECTION

La date est convertie avec `datetime.strptime` puis comparée à `datetime.now()`. Le contrôle est présent dans `/book` et dans `/purchasePlaces`, avec HTTP 400 et le message « Vous ne pouvez pas réserver pour une compétition déjà passée. ».

IMPACT

Respect du cycle de vie des compétitions, cohérence métier et protection contre un contournement par URL directe.

Tests : `test_reservation_competition_passee_refusee_via_purchase`, `test_reservation_competition_passee_refusee_via_book`, `test_reservation_competition_future_autorisee` — **Fichiers :** `server.py`, `competitions.json`.

B9 HTML invalide et formulaires incomplets

MOYEN

SYMPTÔME

Certaines pages contenaient des balises invalides comme `</br>`, des identifiants vides et des formulaires sans attributs `required`, `min` ou `max`. L’accessibilité et la validation navigateur étaient insuffisantes.

CAUSE RACINE

Les templates de démonstration n’avaient pas été conçus pour l’accessibilité ni pour la validation HTML5.

CORRECTION

Les templates ont été nettoyés : balises valides, labels et identifiants uniques, attributs `required`, `min="1"`, `max="12"`, actions de formulaire correctes et structure sémantique plus claire.

IMPACT

Moins de saisies invalides, meilleure expérience pour les utilisateurs et conformité HTML de base pour la revue.

Tests : `test_index_template_attributes`, `test_booking_template_attributes`, `test_welcome_template_clean_html` — **Fichiers :** `templates/index.html`, `templates/booking.html`, `templates/welcome.html`.

B10 Types JSON incohérents

MAJEUR

SYMPTÔME

Les points et les places pouvaient être chargés comme des chaînes de caractères. Les calculs produisaient alors des concaténations, des comparaisons incorrectes ou des erreurs de type.

CAUSE RACINE

Les fichiers JSON de démonstration contenaient des nombres sous forme de texte, et le chargement ne normalisait pas les types.

CORRECTION

`loadClubs()` convertit les points en `int`, `loadCompetitions()` convertit les places en `int` et l'état des inscriptions en `bool`. La réservation conserve des entiers après calcul.

IMPACT

Calculs fiables, comparaisons correctes et compatibilité avec des fichiers JSON hétérogènes.

Tests : `test_load_clubs_points_sont_entiers`, `test_load_competitions_places_sont_entiers`, `test_load_competitions_inscriptions_sont_booleennes`, `test_reservation_maintient_types_entiers` — **Fichiers :** `server.py`, `clubs.json`, `competitions.json`.

SECTION 5 · SUITE ET FIN

5. Résolution des bugs (suite et fin)

B11 Déconnexion sans purge de session

MAJEUR

SYMPTÔME

La déconnexion redirigeait vers l'accueil mais ne confirmait pas l'action et pouvait laisser des informations de session en mémoire.

CAUSE RACINE

La route `/logout` se contentait de rediriger, sans vider la session ni afficher de message.

CORRECTION

La route appelle désormais `session.clear()`, ajoute un message flash « Vous avez été déconnecté avec succès. » puis redirige vers l'accueil.

IMPACT

Confidentialité renforcée, comportement attendu et confirmation claire pour l'utilisateur.

Tests : `test_logout_redirection`, `test_logout_message_flash`, `test_logout_purge_session` — **Fichiers :** `server.py`, `templates/index.html`.

B12 Non-conformités PEP8 et Flake8

MINEUR

SYMPTÔME

Flake8 signalait des lignes trop longues, des espacements irréguliers, des imports mal organisés et quelques constructions peu lisibles.

CAUSE RACINE

Le code de démonstration n'avait pas été formaté selon PEP8, et les ajouts successifs avaient accentué les écarts.

CORRECTION

Le code a été refactorisé : retours à la ligne, espacements, imports, variables et messages. Le rapport final ne contient plus aucune erreur.

IMPACT

Lisibilité, maintenance et crédibilité professionnelle du rendu. Flake8 peut être intégré à une future CI.

Preuve : `flake8 server.py tests generate_flake8_report.py` retourne 0 erreur ; rapport `rapport_flake8.html` — **Fichiers :** `server.py`, `tests/`.

B13 BOM UTF-8 dans `admin.json`

MOYEN

SYMPTÔME

Un fichier `admin.json` enregistré avec un BOM UTF-8 provoquait une erreur de décodage JSON au moment de la connexion administrateur.

CAUSE RACINE

Le fichier était lu avec l'encodage `utf-8`, qui conserve le BOM comme caractère invisible en début de flux, ce que le parseur JSON refuse.

CORRECTION

Tous les chargements utilisent désormais `utf-8-sig`, qui accepte les fichiers avec ou sans BOM. Le fichier `admin.json` du dépôt est également normalisé sans BOM.

IMPACT

Compatibilité avec les éditeurs Windows, disparition d'une erreur difficile à diagnostiquer et robustesse des fichiers de configuration.

Tests : `test_load_admins_accepts_utf8_bom` — **Fichiers :** `server.py`, `admin.json`.

B14 Réservations non persistées après redémarrage

AMÉLIORATION

SYMPTÔME

Les points et les places étaient modifiés en mémoire, mais les fichiers JSON n'étaient pas sauvegardés après une réservation. Après redémarrage, la réservation disparaissait et le classement par compétition devenait faux.

CAUSE RACINE

La route `purchasePlaces` ne faisait aucun appel à `saveClubs()` ou `saveCompetitions()`, et aucun champ ne conservait le cumul des places réservées.

CORRECTION

Le champ `booked` a été ajouté à chaque club. Après une réservation validée, les deux fichiers sont sauvegardés. Une fixture de test empêche les écritures réelles pendant les tests.

IMPACT

Durabilité des réservations, classement par compétition fiable et base nécessaire au contrôle de la limite de 12 places.

Tests : `test_reservation_enregistre_places_et_donnees`, `test_save_functions_write_json` — **Fichiers :** `server.py`, `tests/conftest.py`, `clubs.json`.

B15 Classement général non trié

AMÉLIORATION

SYMPTÔME

Le tableau public affichait les clubs dans l'ordre du fichier JSON, et non dans l'ordre des points. Le classement n'avait donc pas de valeur comparative fiable.

CAUSE RACINE

La route `displayBoard` transmettait la liste des clubs telle quelle au template, sans tri.

CORRECTION

Les clubs sont triés par points décroissants, puis par nom en cas d'égalité. Ce tri est appliqué au classement général ; le classement par compétition possède son propre tri par places réservées.

IMPACT

Classement lisible et équitable, cohérent avec l'objectif de transparence du projet.

Tests : `test_display_board_clubs_sorted_by_points`, `test_display_board_renders_all_clubs_and_points` — **Fichiers :** `server.py`, `templates/display_board.html`.

Bilan des corrections. Les 13 bugs et les 2 améliorations sont couverts par des tests de non-régression. Les cas d'erreur ne modifient ni les points ni les places, et chaque correction a été vérifiée avec la suite complète.

SECTION 6

6. Traçabilité des exigences

Ce tableau relie chaque exigence du cahier des charges ou chaque fonctionnalité complémentaire à son implémentation et à sa preuve automatisée. Il permet de vérifier rapidement que le périmètre livré n'est pas seulement déclaratif.

Exigence	Implémentation	Preuve	Statut
Connexion du secrétaire par e-mail	`/showSummary`	`test_connexion_reussie`	Conforme
Message propre pour un e-mail inconnu	`/showSummary` + HTTP 401	`test_connexion_email_inconnu`, `test_connexion_email_vide`	Conforme
Consultation des compétitions et des places	`welcome.html`	`test_welcome_template_clean_html`, `test_welcome_shows_closed_registration`	Conforme
Réservation de places	`/book`, `/purchasePlaces`	`test_deduction_points_apres_reservation`	Conforme
Déduction des points	`purchasePlaces`	`test_deduction_points_apres_reservation`	Conforme
Confirmation de réservation	Message flash « Great-booking complete! »	`test_deduction_points_apres_reservation`	Conforme
Refus si points insuffisants	HTTP 400 dans `purchasePlaces`	`test_reservation_points_insuffisants`	Conforme
Maximum 12 places par club et compétition	`booked` + HTTP 400	`test_limite_places.py`	Conforme
Refus si compétition pleine	Contrôle de `numberOfPlaces`	`test_surreservation_places_competition_refusee`	Conforme
Refus des compétitions passées	Comparaison de dates	`test_competitions_passees.py`	Conforme
Déconnexion	`/logout` + `session.clear()`	`test_deconnexion.py`	Conforme
Tableau public accessible sans connexion	`/displayBoard`, `/points`	`test_display_board_public_access`, `test_points_alias_access`	Conforme
Tableau public en lecture seule	`display_board.html` sans formulaire	`test_display_board_public_access` + revue du template	Conforme
Classement général trié par points	Tri dans `displayBoard`	`test_display_board_clubs_sorted_by_points`	Conforme
Classement par compétition	Sélecteur + `booked`	`test_display_board_competition_ranking`	Conforme
Performance : liste < 5 s	Locust, pages légères	`rapport_locust.html`	Conforme
Performance : points < 2 s	Locust, traitement local	`rapport_locust.html`	Conforme
Couverture de tests ≥ 60 %	94 tests, couverture 100 %	`htmlcov/index.html`	Conforme
README à jour	`README.md`	Installation, comptes, tests, workflow Git	Conforme
Une branche par correctif / fonctionnalité	Branches `bug/` et `feature/`	`git branch --no-merged main`	Conforme

Exigence	Implémentation	Preuve	Statut
Revue finale	Branche `QA` alignée sur `main`	`git branch -vv`	Conforme
Administration des clubs et compétitions	`/admin/*`	`test_admin.py`	Conforme — bonus
Gestion des points et historique	`adminClubPoints`, `recordPointsChange`	`test_admin_points.py`	Conforme — bonus
Ouverture / fermeture des inscriptions	`registration_open`	`test_admin_inscriptions.py`	Conforme — bonus
Persistance des réservations	`booked` + sauvegardes	`test_reservation_enregistre_places_et_donnees`	Conforme — bonus

Conclusion de traçabilité. Chaque exigence identifiée possède une implémentation vérifiable et au moins une preuve automatisée ou un rapport. Les fonctionnalités bonus sont également couvertes par des tests.

SECTION 7

7. Qualité, performance et sécurité

7.1 Inventaire des tests

Fichier de test	Nombre	Ce qui est vérifié
`test_admin.py`	36	Connexion admin, protection des routes, CRUD clubs et compétitions, BOM, sauvegardes.
`test_admin_inscriptions.py`	9	Bascule ouverture / fermeture, blocage 403, affichage club, valeurs par défaut.
`test_admin_points.py`	9	Crédit, débit, solde négatif, montant invalide, motif, historique.
`test_club_competition_inconnus.py`	5	Clubs et compétitions inconnus sur `/book` et `/purchasePlaces`.
`test_competitions_passees.py`	3	Refus des compétitions passées et acceptation d'une compétition future.
`test_connexion.py`	3	Connexion valide, e-mail inconnu, e-mail vide.
`test_deconnexion.py`	3	Redirection, message flash, purge de session.
`test_limite_places.py`	3	Limite de 12, cumul de réservations, surréservation.
`test_points.py`	3	Déduction des points, persistance, points insuffisants.
`test_points_display.py`	10	Accès public, tri, sélecteur, classement par compétition, état vide, alias.
`test_saisie_places.py`	3	Places négatives, nulles, non numériques.
`test_templates_html.py`	3	Attributs des formulaires, accessibilité de base, HTML propre.
`test_types_donnees.py`	4	Points et places entiers, inscriptions booléennes, types après réservation.
Total	**94**	**Couverture fonctionnelle et technique du projet.**

7.2 Couverture et conformité

94/94
tests réussis

100 %
couverture de `server.py`

0
erreur Flake8

La couverture de `server.py` atteint 100 %, bien au-delà du seuil de 60 % demandé. Le rapport HTML est disponible dans `htmlcov/index.html`. Flake8 ne signale aucune erreur sur le serveur, les tests et le script de rapport. Le rapport lisible est disponible dans `rapport_flake8.html`.

7.3 Performance et charge

Indicateur Locust	Résultat	Seuil ou lecture
Utilisateurs simulés	6	Conforme au cahier des charges
Durée	30 secondes	Test court de validation
Requêtes	97	0 échec
Temps moyen	2 ms	Très inférieur aux seuils

Indicateur Locust	Résultat	Seuil ou lecture
95e percentile	4 ms	Liste < 5 s, points < 2 s
Temps maximum	7 ms	Aucun pic problématique

Le rapport complet est disponible dans `rapport_locust.html`. Sous Windows, Locust 1.6.0 peut afficher un message lié à la console (`SetConsoleMode`) sans que cela remette en cause les résultats du test.

7.4 Git, branches et revue

Le projet suit un workflow de branches courtes : une branche par correctif ou fonctionnalité, des commits explicites, puis un merge dans `main` avec `--no-ff`. Les merges squash et rebase ne sont pas utilisés, afin de conserver la trace des branches et des décisions.

```
*   99372e7 Merge : classement public par competition
|\
| * 8e5f975 Feature : classement public par competition
* |   2e2197c Merge : gestion des points et des inscriptions
|\ \
| * 06cd9b0 Feature : gestion des points et des inscriptions
|/
*   1535974 Fix : chargement de admin.json avec BOM UTF-8
```

Branche	Rôle	État
`main`	Branche de livraison.	À jour
`QA`	Miroir de `main` pour la revue finale, non mergé dans `main`.	Alignée
`feature/tableau-public-points-clubs`	Classement public général.	Mergée
`feature/tableau-de-bord-admin`	Administration et tableau de bord.	Mergée
`feature/gestion-points-et-inscriptions`	Points, historique et ouverture des inscriptions.	Mergée
`feature/classement-public-par-competition`	Classement par compétition.	Mergée
`bug/*`	Correctifs unitaires (connexion, points, 12 places, dates, HTML, types, etc.).	Toutes mergées

Les commandes de vérification sont `git branch --no-merged main` et `git branch -r --no-merged main` : elles ne retournent aucune branche non mergée. La branche `QA` peut être réalignée avec `git branch -f QA main` puis `git push --force-with-lease origin QA`.

SECTION 7 · SUITE

7. Qualité, performance et sécurité (suite)

7.5 Sécurité : ce qui est déjà protégé

- **Routes admin protégées** par le décorateur `admin_required` : une session administrateur valide est nécessaire.
- **Déconnexion** avec `session.clear()`, qui supprime les informations de session.
- **Validation serveur** systématique, même si la validation HTML5 est contournée.
- **Codes HTTP explicites** : 400, 401, 403 et 404 au lieu d'erreurs 500.
- **Contrôle des points et des places** avant toute mutation : aucune réservation invalide ne modifie les fichiers.
- **Tests isolés** : une fixture `autouse` neutralise les sauvegardes JSON pour ne pas altérer les données réelles.
- **Lecture tolérante au BOM** avec `utf-8-sig`, y compris pour `admin.json`.

7.6 Points de vigilance

Point	État actuel	Recommandation
Clé secrète Flask	Valeur de démonstration dans le code.	Variable d'environnement et clé aléatoire longue.
Mot de passe admin	Stocké en clair dans `admin.json`.	Hachage avec bcrypt ou argon2.
Protection CSRF	Non implémentée.	Jetons CSRF sur les formulaires admin et club.
Limitation de débit	Non implémentée.	Flask-Limiter sur la connexion et les actions sensibles.
Stockage	Fichiers JSON.	Base SQLite / PostgreSQL avec transactions.
Écritures concurrentes	Pas de verrou ni d'écriture atomique.	Fichier temporaire + remplacement atomique, ou base de données.
HTTPS	Serveur de développement HTTP.	Reverse proxy HTTPS en production.
Authentification club	Identification par e-mail, sans session protégée sur les routes de réservation.	Session club signée et contrôle d'accès aux routes `/book` et `/purchasePlaces`.
Rôles	Un seul rôle administrateur.	Rôles et permissions granulaires si nécessaire.
Audit global	Historique des points uniquement.	Journaliser aussi les CRUD clubs et compétitions.

> **Positionnement.** L'application est solide pour une démonstration, une formation ou une recette fonctionnelle. Elle n'est pas prête pour une exposition publique en production sans le durcissement ci-dessus.

SECTION 8

8. Limites et pistes d’amélioration

8.1 Limites connues

Limite	Impact	Contournement actuel
Stockage JSON sans transaction	Risque de perte ou d’incohérence en cas d’écriture concurrente.	Usage mono-utilisateur et tests isolés.
Pas de base de données	Pas de requêtes, d’index ni de contraintes d’intégrité.	Volumes faibles de démonstration.
Pas de hachage des mots de passe	Un fichier `admin.json` exposé révèle les identifiants.	Compte de démonstration uniquement.
Pas de CSRF	Un formulaire malveillant pourrait soumettre une action admin.	Usage local et revue de code.
Pas de pagination admin	Listes longues moins confortables.	Jeu de données réduit.
Pas de contrôle d’unicité	Doublons de clubs, e-mails ou compétitions possibles.	Validation manuelle par l’admin.
Pas de journal d’audit global	Les CRUD ne sont pas historisés.	Historique Git et historique des points.
Tests dans un seul dossier	Pas de séparation unitaire / intégration / fonctionnel.	Tests Flask couvrant routes et templates.

8.2 Améliorations prioritaires

Priorité	Amélioration	Bénéfice attendu
P1	Hacher les mots de passe et externaliser la clé secrète.	Sécurité de base.
P1	Ajouter CSRF et limitation de débit.	Protection contre les abus et les soumissions forgées.
P2	Migrer vers SQLite ou PostgreSQL avec transactions.	Intégrité, concurrence et requêtes.
P2	Écritures atomiques des fichiers JSON si le stockage est conservé.	Réduction du risque de corruption.
P3	Intégrer une CI GitHub Actions : tests, couverture, Flake8.	Détection automatique des régressions.
P3	Séparer tests unitaires, d’intégration et fonctionnels.	Lisibilité et granularité des diagnostics.
P4	Pagination, recherche et tri dans l’administration.	Confort sur des volumes plus importants.
P4	Audit accessibilité (contrastes, navigation clavier, lecteurs d’écran).	Conformité et inclusion.
P5	Journalisation applicative et supervision.	Exploitation et diagnostic en production.

8.3 Ce qui est déjà solide

- Les règles métier critiques sont appliquées côté serveur, pas seulement dans l’interface.
- Les cas d’erreur ne modifient ni les points ni les places.
- Les fonctionnalités sont couvertes par des tests de non-régression.

- La séparation entre routes, templates et données reste lisible.
- La documentation et l'historique Git permettent une reprise rapide.

8.4 Checklist avant une mise en production

Point de contrôle	Attendu	État actuel
Clé secrète Flask	Variable d'environnement, valeur aléatoire longue.	À faire
Mot de passe admin	Haché avec bcrypt ou argon2.	À faire
Protection CSRF	Jetons sur tous les formulaires sensibles.	À faire
HTTPS	Reverse proxy et certificat valide.	À faire
Stockage	Base transactionnelle et sauvegardes.	À faire
Supervision	Logs, métriques et alertes.	À faire
Tests automatisés	CI qui exécute pytest, couverture et Flake8.	Tests locaux prêts
Documentation	README, rapports et procédure de restauration.	Documentation prête

Cette checklist distingue clairement l'état de démonstration actuel d'une cible de production. Elle peut servir de support à une phase de durcissement ou à un futur sprint technique.

SECTION 9

9. Conclusion

Le projet GUDLFT ne se limite pas à un exercice de correction : il transforme une application de démonstration fragile en une base fonctionnelle, testée, administrable et transparente.

Les fonctionnalités ajoutées répondent à des besoins concrets. Le classement public restaure la confiance entre les clubs. Le classement par compétition donne du contexte à la participation. L'administration libère l'organisateur de l'édition manuelle des fichiers JSON. La gestion des points apporte une traçabilité attendue. L'ouverture et la fermeture des inscriptions donnent un contrôle opérationnel simple. La persistance garantit que les décisions ne disparaissent pas au redémarrage.

Les corrections de bugs ont supprimé les erreurs serveur les plus visibles, appliqué les règles de points, de capacité, de date et de limite de 12 places, et normalisé les données. Chaque correction est associée à un test, ce qui protège les fonctionnalités existantes et facilite les évolutions.

100 % du périmètre fonctionnel identifié	**94** tests de non-régression	**0** échec Locust
0 erreur Flake8	**100 %** couverture de server.py	**0** branche non mergée

Ce qu'il faut retenir

- **Transparence :** les points et les classements sont consultables publiquement.
- **Autonomie :** l'organisateur gère clubs, compétitions, points et inscriptions depuis l'interface.
- **Fiabilité :** les règles métier sont vérifiées côté serveur et couvertes par des tests.
- **Traçabilité :** l'historique des points, les branches et les commits de merge documentent les décisions.
- **Maintenabilité :** code conforme PEP8, structure lisible, README et rapports HTML.

> **Statut final.** Le projet est prêt pour la démonstration et la revue finale. Les éventuels ajustements de recette pourront être traités sur une nouvelle branche, puis mergés avec `--no-ff` dans `main`, avant réalignement de `QA`.

ANNEXE A

Annexe A — Routes de l'application

Les routes sont regroupées en trois familles : accès public, parcours club et administration. Les routes admin sont protégées par `admin_required` ; les routes club s'appuient sur l'e-mail saisi pour identifier le club, sans session club protégée.

Méthode	Route	Accès	Rôle
GET	`/`	Public	Page de connexion club.
POST	`/showSummary`	Public	Identification du club par e-mail et tableau de bord.
GET	`/book/<competition>/<club>`	Club	Formulaire de réservation.
POST	`/purchasePlaces`	Club	Validation de la réservation, déduction des points et des places.
GET	`/displayBoard`	Public	Classement général ou classement par compétition.
GET	`/points`	Public	Alias du classement public.
GET	`/logout`	Club	Déconnexion, purge de session et message flash.
GET, POST	`/admin/login`	Public	Connexion administrateur.
GET	`/admin/logout`	Admin	Déconnexion administrateur.
GET	`/admin`	Admin	Tableau de bord et statistiques.
GET	`/admin/clubs`	Admin	Liste des clubs.
GET, POST	`/admin/clubs/add`	Admin	Ajout d'un club.
GET, POST	`/admin/clubs/edit/<club_name>`	Admin	Modification d'un club.
GET, POST	`/admin/clubs/<club_name>/points`	Admin	Gestion des points et historique.
POST	`/admin/clubs/delete/<club_name>`	Admin	Suppression d'un club.
GET	`/admin/competitions`	Admin	Liste des compétitions.
GET, POST	`/admin/competitions/add`	Admin	Ajout d'une compétition.
GET, POST	`/admin/competitions/edit/<comp_name>`	Admin	Modification d'une compétition.
POST	`/admin/competitions/<comp_name>/toggle-registration`	Admin	Ouverture ou fermeture des inscriptions.
POST	`/admin/competitions/delete/<comp_name>`	Admin	Suppression d'une compétition.

ANNEXE B

Annexe B — Modèle de données

B.1 `clubs.json`

Champ	Type	Description
`name`	chaîne	Nom unique du club, utilisé dans les URLs et les classements.
`email`	chaîne	Adresse utilisée pour identifier le club à la connexion.
`points`	entier	Solde de points disponible. Un point vaut une place.
`points_history`	liste	Historique des ajustements admin : date, admin, montant, motif, solde.
`booked`	objet	Nombre de places réservées par compétition : `{"Future Open": 3}`.

B.2 `competitions.json`

Champ	Type	Description
`name`	chaîne	Nom de la compétition.
`date`	chaîne	Date au format `YYYY-MM-DD HH:MM:SS`.
`numberOfPlaces`	entier	Places restantes. Décrémenté à chaque réservation.
`registration_open`	booléen	État des inscriptions ; vrai par défaut.

B.3 `admin.json`

Champ	Type	Description
`email`	chaîne	Identifiant de connexion administrateur.
`password`	chaîne	Mot de passe de démonstration, à hacher en production.
`name`	chaîne	Nom affiché dans l'interface et l'historique des points.

B.4 Exemple d'entrée d'historique

```
{
  "date": "2026-10-25 10:15:00",
  "admin": "Super Admin",
  "amount": 5,
  "reason": "Bonus partenaire",
  "balance": 18
}
```

Les champs **`points_history`** et **`booked`** sont créés à la demande s'ils sont absents. Les dates sont comparées avec **`datetime.strptime`** et le format défini dans les données.

ANNEXE C

Annexe C — Tests représentatifs

La suite complète contient 94 tests. Le tableau ci-dessous présente les tests les plus représentatifs par domaine, afin de faciliter la revue sans lister les 94 noms.

Domaine	Tests représentatifs	Attendu
Connexion	`test_connexion_reussie`, `test_connexion_email_inconnu`, `test_connexion_email_vide`	200, 401 et 401 avec messages.
Réservation	`test_deduction_points_apres_reservation`, `test_reservation_enregistre_places_et_donnees`	Points, places et `booked` mis à jour ; sauvegardes appelées.
Points insuffisants	`test_reservation_points_insuffisants`	400 et aucun changement.
Limite de 12	`test_reservation_superieure_a_12_places_refusee`, `test_reservation_cumulee_superieure_a_12_places_refusee`	400, cumul bloqué.
Surréservation	`test_surreservation_places_competition_refusee`	400, stock inchangé.
Saisie invalide	`test_reservation_places_negatives_refusee`, `test_reservation_zero_place_refusee`, `test_reservation_places_non_numeriques_refusee`	400 sans crash.
Dates	`test_reservation_competition_passee_refusee_via_purchase`, `test_reservation_competition_passee_refusee_via_book`, `test_reservation_competition_future_autorisee`	400 pour le passé, 200 pour le futur.
Inconnus	`test_book_club_inconnu`, `test_book_competition_inconnue`, `test_purchase_club_inconnu`	404 avec messages.
Classement public	`test_display_board_public_access`, `test_display_board_clubs_sorted_by_points`, `test_display_board_competition_ranking`	200, tri et classement par compétition.
Administration	`test_admin_routes_require_login`, `test_admin_login_success`, tests CRUD	Protection, création, modification, suppression.
Points admin	`test_admin_credit_points`, `test_admin_debit_points`, `test_admin_debit_cannot_make_balance_negative`	Solde et historique cohérents.
Inscriptions	`test_admin_toggle_registration_closes_then_reopens`, `test_closed_competition_blocks_purchase`	Bascule et blocage 403.
Types et BOM	`test_load_clubs_points_sont_entiers`, `test_load_admins_accepts_utf8_bom`	Types normalisés, BOM accepté.
Déconnexion	`test_logout_redirection`, `test_logout_message_flash`, `test_logout_purge_session`	302, message, session vide.

C.1 Tests non fonctionnels

Vérification	Commande	Résultat
Tests et couverture	`python -m pytest --cov=server --cov-report=html`	94 tests, 100 %
Style PEP8	`flake8 server.py tests generate_flake8_report.py`	0 erreur
Charge	`locust -f locustfile.py --headless -u 6 -r 1 -t 30s`	0 échec, p95 4 ms

C.2 Ce que les tests ne couvrent pas

- **La concurrence réelle :** les tests utilisent un seul client Flask ; les écritures JSON simultanées ne sont pas simulées.
- **La sécurité offensive :** CSRF, force brute, injection et mauvaise configuration ne sont pas testés automatiquement.
- **L’accessibilité complète :** les attributs de base sont vérifiés, mais pas la navigation clavier ni les lecteurs d’écran.
- **Le rendu visuel :** les tests vérifient le HTML et les comportements, pas la qualité graphique sur tous les navigateurs.
- **La production :** HTTPS, base de données, sauvegardes et supervision sont hors du périmètre de cette démonstration.

Ces limites sont assumées et documentées. Elles n’empêchent pas la validation fonctionnelle, mais elles définissent le travail restant avant une exposition publique.

ANNEXE D

Annexe D — Commandes utiles

D.1 Installation

```
python -m venv env
.\env\Scripts\Activate.ps1
pip install -r requirements.txt
```

D.2 Lancement de l'application

```
$env:FLASK_APP = "server"
$env:FLASK_ENV = "development"
flask run
```

Adresses : `http://127.0.0.1:5000`, `http://127.0.0.1:5000/displayBoard` et `http://127.0.0.1:5000/admin/login`.

D.3 Tests et couverture

```
pytest tests/
pytest --cov=server tests/
pytest --cov=server --cov-report=html tests/
```

Le rapport de couverture HTML est généré dans `htmlcov/index.html`.

D.4 Flake8

```
flake8 server.py tests/ locustfile.py generate_flake8_report.py --count --statistics --show-source
python generate_flake8_report.py
```

Le rapport lisible est généré dans `rapport_flake8.html`.

D.5 Locust

```
locust -f locustfile.py --host http://127.0.0.1:5000
```

Version sans interface web :

```
locust -f locustfile.py --headless -u 6 -r 1 -t 30s `
  --host http://127.0.0.1:5000 --html=rapport_locust.html
```

D.6 Workflow Git

```
git switch main
git pull origin main
git switch -c feature/nom-de-la-fonctionnalite

# developpement, tests, commits
git add .
git commit -m "Feature : description courte"
git push -u origin feature/nom-de-la-fonctionnalite

git switch main
git pull origin main
git merge --no-ff feature/nom-de-la-fonctionnalite `
  -m "Merge : nom-de-la-fonctionnalite"
git push origin main

# alignement de la branche de revue
git branch -f QA main
git push --force-with-lease origin QA
```

Ne pas utiliser **Squash and merge** ni **Rebase and merge** pour les fonctionnalités : l'historique doit conserver les commits de merge.

ANNEXE E

Annexe E — Glossaire

Club

Structure qui inscrit des athlètes et dispose d'un solde de points.

Secrétaire de club

Utilisateur qui se connecte avec l'e-mail du club pour réserver des places.

Compétition

Événement daté avec un nombre de places disponibles et un état d'inscriptions.

Place

Unité réservable dans une compétition. Une place coûte un point.

Point

Monnaie interne du club, gagnée en organisant des compétitions et dépensée en réservations.

Inscriptions

État d'une compétition qui autorise ou bloque les réservations.

Classement général

Liste des clubs triée par points décroissants, puis par nom.

Classement par compétition

Liste des clubs ayant réservé des places dans une compétition, triée par places réservées.

`booked`

Dictionnaire qui associe une compétition au nombre de places déjà réservées par un club.

`registration_open`

Booléen qui indique si les inscriptions sont ouvertes.

`points_history`

Liste horodatée des ajustements de points effectués par un administrateur.

QA

Branche de revue alignée sur `main`, non mergée dans `main`.

Merge commit

Commit qui conserve explicitement l'intégration d'une branche dans `main`.

Couverture

Pourcentage du code exécuté par les tests. Ici, 100 % de `server.py`.

Flake8

Outil de contrôle de style et de conformité PEP8.

Locust

Outil de test de charge utilisé avec 6 utilisateurs simulés.

BOM

Marqueur invisible en début de fichier UTF-8, géré avec `utf-8-sig`.

ANNEXE F

Annexe F — Comptes de démonstration

Ces comptes sont destinés à la démonstration et à la recette locale. Ils ne doivent pas être conservés tels quels dans un déploiement réel.

F.1 Clubs

E-mail	Club
`john@simplylift.co`	Simply Lift
`admin@irontemple.com`	Iron Temple
`kate@shelifts.co.uk`	She Lifts

F.2 Administration

Champ	Valeur
URL	`http://127.0.0.1:5000/admin/login`
E-mail	`admin@gudlft.com`
Mot de passe	`admin1234`

> **À faire avant tout déploiement.** Changer le mot de passe administrateur, externaliser la clé secrète Flask, ajouter CSRF, limiter les tentatives de connexion et migrer vers un stockage transactionnel.

F.3 URLs utiles

Page	URL locale
Application club	`http://127.0.0.1:5000/`
Classement public	`http://127.0.0.1:5000/displayBoard`
Classement par compétition	`http://127.0.0.1:5000/displayBoard?competition=Future%20Open`
Administration	`http://127.0.0.1:5000/admin/login`

Fin du document — GUDLFT, documentation fonctionnelle et technique, version 1.0, 25 octobre 2026. Document préparé pour la revue finale et la démonstration du projet.